Security in Computing and Information Technology

Assignment 2

Jasanjot Singh (s4093354) and Kyle Bonjour (s4003110)

1)

Generating public (n,g) and private (λ,μ) keys:

p: 107, q: 61, g: 7019

n=pq

n = 107 * 61

n = 6,527

public key: (6527,7019)

λ = lcm(p-1,q-1)

λ = lcm(107-1, 61-1) → λ = lcm(106,60)

λ = 3180

μ = $(L(g^{\lambda} mod n^2)^{-1} mod * n$ → u = $7019^{3180} mod 6527^2$

u = 33568362

L(u) = $\frac{u-1}{n}$

L = $\frac{33568362-1}{6527}$

L = 5143

μ = $5143^{-1} mod 6527$

μ = 6145

private key: (3180,6145)

Encryption of Individual Votes Using the Paillier Cryptosystem

Table-1: Votes of 10 voters

Voter No.	Voter's Private Number, *r*	Vote for	Voting message, *m*
1	*r1=71*	ALICE	000100000000 = 256
2	*r2=72*	ALICE	000100000000 = 256
3	*r3=73*	ALICE	000100000000 = 256
4	*r4=74*	ALICE	000100000000 = 256
5	*r5=75*	BOB	000000010000 = 16
6	*r6=76*	BOB	000000010000 = 16
7	*r7=77*	BOB	000000010000 = 16
8	*r8=78*	EVE	000000000001 = 1
9	*r9=79*	EVE	000000000001 = 1
10	*r10=80*	EVE	000000000001 = 1

$$c_1 = g^{m_1} r_1{}^n mod n^2$$

$$c_1 = 7019^{256} 71^{6527} mod 6527^2$$

$$c_1 = 22787030$$

$$c_2 = g^{m_2} r_2{}^n mod n^2$$

$$c_2 = 7019^{256} 72^{6527} mod 6527^2$$

$$c_2 = 6155676$$

$$c_3 = g^{m_3} r_3{}^n mod n^2$$

$$c_3 = 7019^{256} 73^{6527} mod 6527^2$$

$$c_3 = 26435665$$

$$c_4 = g^{m_4} r_4{}^n mod n^2$$

$$c_4 = 7019^{256} 74^{6527} mod 6527^2$$

$$c_4 = 5713316$$

$$c_5 = g^{m_5} r_5{}^n mod n^2$$

$$c_5 = 7019^{16} 75^{6527} mod 6527^2$$

$$c_5 = 35730009$$

$$c_6 = g^{m_6} r_6{}^n mod n^2$$

$$c_6 = 7019^{16} 76^{6527} mod 6527^2$$

$$c_6 = 22037180$$

$$c_7 = g^{m_7} r_7{}^n mod n^2$$

$$c_7 = 7019^{16} 75^{6527} mod 6527^2$$

$$c_7 = 30448834$$

$$c_8 = g^{m_8} r_8{}^n mod n^2$$

$$c_8 = 7019^{1}75^{6527} mod 6527^2$$

$$c_8 = 35591$$

$$c_9 = g^{m_9} r_9{}^n mod n^2$$

$$c_9 = 7019^{1}75^{6527} mod 6527^2$$

$$c_9 = 31639803$$

$$c_{10} = g^{m_{10}} r_{10}{}^n mod n^2$$

$$c_{10} = 7019^{1}75^{6527} mod 6527^2$$

$$c_{10} = 28502453$$

Homomorphic Aggregation of Encrypted Votes

C = $c_1 * c_2 * c_3 * c_4 * c_5 * c_6 * c_7 * c_8 * c_0 * c_{10} mod 6527^2$

$$\begin{aligned} C = (&22\,787\,030 * 6\,155\,676 * 26\,435\,665 \\ &* 5\,713\,316 * 35\,730\,009 * 22\,037\,180 \\ &* 30\,448\,834 * 35\,591 * 31\,639\,803 \\ &* 28\,502\,453) \bmod 6527^2 \end{aligned}$$

C = 30566829

Decryption of encrypted tally

u = $C^{\lambda} mod n^2$ → u = $30566829^{3180} mod 6527^2$

u = 2323613

L(u) = $\frac{u-1}{n}$ → L(u) = $\frac{2323613-1}{6527}$

L(u) = 356

M = L(u) * μ mod n → M = (356 * 6145) mod 6527

M = 1075

Conversion of decrypted value into 12 bit binary

The decrypted tally, (1075), is converted into binary by successively subtracting the largest power of two that does not exceed the remaining value:

1075 - 1024 = 51

51 – 32 = 19

19 – 16 = 3

3 – 2 = 1

1 – 1 = 0

1075 = 1024 + 32 + 16 + 2 + 1

A bit value of 1 is assigned to each power of two **used** in the sum, while 0 is assigned to each **unused** power

Power of 2	**2048**	**1024**	**512**	**256**	**128**	**64**	**32**	**16**	**8**	**4**	**2**	**1**
Bit	0	1	0	0	0	0	1	1	0	0	1	1

$$1075_{10} = 010000110011_2$$

$$1075_{10} = 0100\ 0011\ 0011_2$$

Reading the three 4-bit fields in the order Alice, Bob, and Eve gives 0100_2=4, 0011_2=3, and 0011_2=3, respectively. Therefore, Alice receives 4 votes, Bob receives 3 votes, and Eve receives 3 votes. The total is 10 votes, and Alice is elected club president

2) code on vs studio code

3) code on vs studio code

4) code on vs studio code

5) Side note: in case link does not work, the video is also in the assignment2 zip folder which was submitted under "**q4 video demo.mp4"**

Call with Kyle Bonjour-20261019_223954-Meeting Recording.mp4

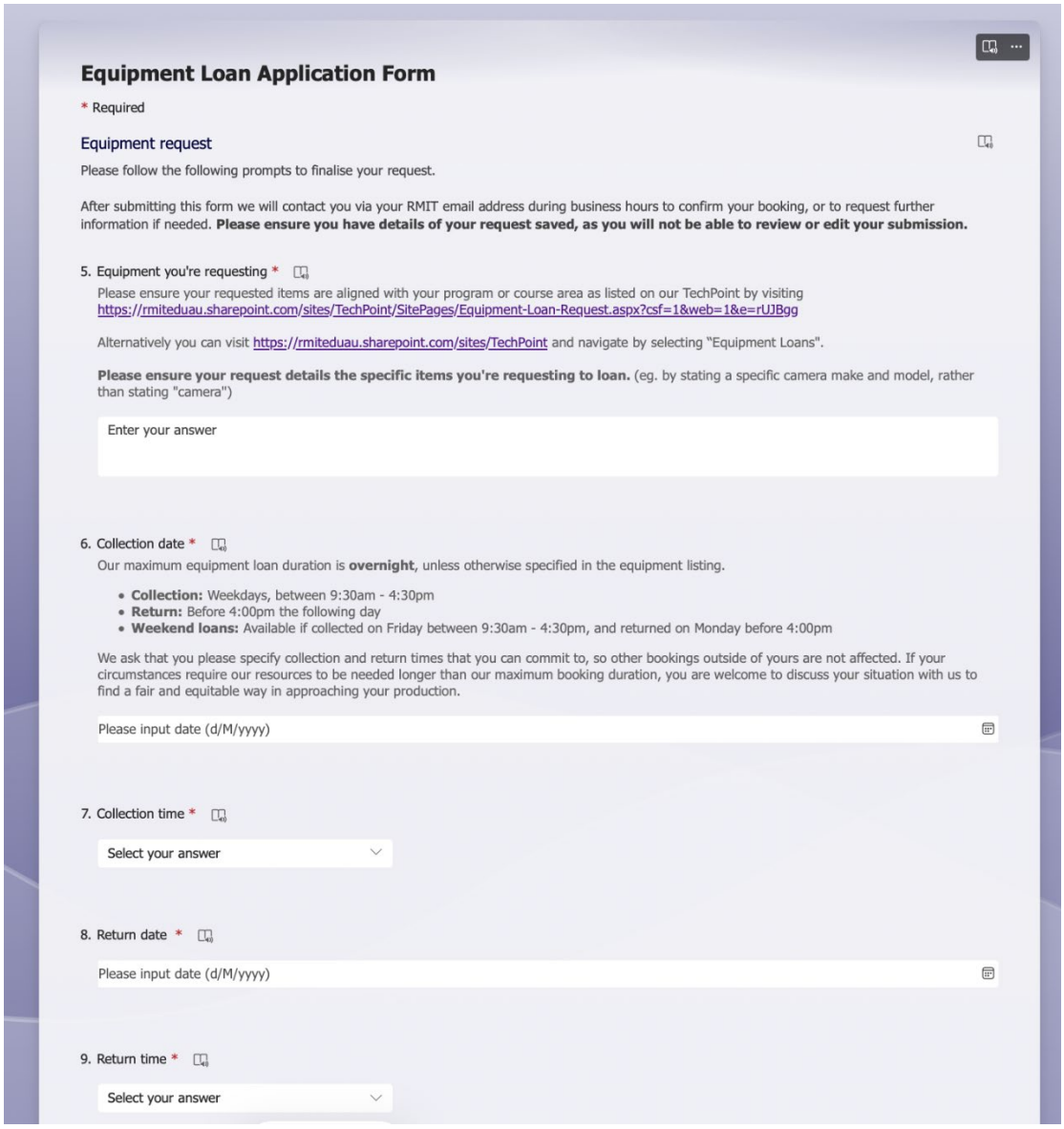
6a)

i. Significance (overview and scenarios)

When starting a creative project a writer dreams and imagines a world to escape theirs and captures moments and scenes that will draw a view into that world. What can hold a writer back is lack of equipment but RMIT offers a service to specific students within certain courses to loan equipment to capture what they could not before. The application form that currently exists is outdated and sees the user filling out a text field with what they desire to loan for their project. There is a separate website that is attached in the form that allows the user to see what equipment they can loan based on their degree but the condition of the items and availability are not stated and can only be found after submission of the form.

ii. Background (existing solutions)

The existing system is a microsoft form that is filled out by the student, the form consists of three pages, the first is what course the equipment is being used for. Then the next page requests contact details from the user and then finally the form asks what gear the student wants. The student is asked to be specific about the details for which gear they want the form has just a free-text box to list out each item they want without any reference other than a separate website with a spreadsheet of items, leading to missing items that are required to be stated in the form, they are then requested to fill in the collection date and drop off date and in additional comments. If they want to pick up at a specific location they tell us to specify which collection place then after which the form is submitted and within a few days they get confirmation of the requested gear and the collection date and drop off in accordance to availability and repairs. The form originates from a website called “Techpoint” which uses Microsoft's software sharepoint to showcase all the equipment. However it does not show current or recent photos of the equipment but rather stock photos, this current solution to see all the equipment completely misses the existing damage and condition of the equipment and as well any repairs that have been done to the equipment. This has led to students returning damaged gear and being put to blame for the damage but also the experience of students hiring gear only for it to be in an undesirable condition. An image of the current form is attached below.



iii.

Aims and objectives

When loaning equipment to create a project the process should be simple, RMIT is providing for beginners so helping and assisting in any way possible for new creators is the main objective. The Goal is to design a helpful, simpler and more transparent platform for students to loan gear from RMIT. To build on the helpfulness of this system the free-text input will be replaced by simple selectable items that will be able to be added to a cart also an availability timeframe is given for the selected gear. Users will be prompted with missing items or suggested kits that go with gear they have loaned like headphones or an SD card ensuring nothing is forgotten before submitting a request. To bring transparency to the system condition will be displayed with notes on any damages, as well repair history will be noted down. Within this system, automation can be introduced to reduce staff admin through automated approvals, with certain rules that will allow approvals and red flags that will require human review. As well when gear is assigned to a student the check in and check out times are timestamped into the records so disputes for pick up and drop off times are removed. Within these

Aims and Objectives, it is desired to create a secure, efficient and helpful way for students to loan gear to create their dream projects.

iv. System overview (the core of the report)

The new proposed system will use a private permissioned blockchain. Hyperledger Fabric is an enterprise grade permissioned distributed ledger technology that will allow for RMIT to allow for a private permissioned blockchain, giving them power over participants such as students, equipment staff and any repair technicians. Each item will receive its own on chain record, containing an ID, current photos of the equipment and last owner, the three of these will be stored on chain and off chain, with the on chain data being hashed. The chain will also hold condition information, repair history. For Smart Contracts there will be 4 rules to assist the team with automation. The first rule will be eligibility, allowing only students within specific courses to request items. To ensure there is no double booking availability and booking will be another part of the smart contract. For check in and check out, both the staff and loaning student must sign off on the condition of the equipment, photos will be timestamped to ensure accurate information on condition of equipment. The last rule will be to return reminders for the students, ensuring timely returns for equipment and as well reminding them to sign off on condition when returning. The student facing UI will get a complete overhaul, instead of the form that is currently used there will be an online store look for the loaning process, students will first be prompted to log in using RMIT's SSO and based on their degree they will be able to select items to add to their cart, once all items are selected they are then prompted within their cart to select their required dates from a calendar. The calendar will show availability per item and show the best times for all the equipment that they have selected to be loaned. They are also able to select from selected kits, for example an 'Outdoor Documentary Shoot' including all necessary gear for a specific project without the hassle of selecting each item.

vi. Advantages and limitations

The advantages for this system is complete transparency, students and staff are able to track the condition of the equipment without any dispute on who the damage is done by but a completely transparent system that tracks condition and repairs. There are also automated processes for the staff taking the strain of chasing equipment up but rather have the smart contract automate the work for the equipment staff.

The system being designed is an in-depth and costly system for a very small scale service. The cost being a maintenance staff for this blockchain, cost of the service itself and as well the technical skills that will be required by the staff. RMIT uses Microsoft systems for a lot of processes and to get this proposed system to be integrated with

RMIT systems will be a strenuous task to start up and also there will be concerns with the system making mistakes on start up and the cost of the mistake can be equipment upwards of thousands dollars worth of equipment. A well designed database could fix some of the problems stated within the background of the current solution but implementing blockchain strengthens the integrity of recorded damages with a clear audit trail of damage.

vii. Conclusion

Nothing should hold a student back from their dream project and RMIT has offered a service that is helping students build these dream projects but the process can be simplified with the assistance of Blockchain. Within its current state students are loaning equipment blindly by submitting a form with little to information other than the equipment available to them, leaving students with equipment with undesired condition or with form submission without any confirmation of availability. The proposed solution gives students the freedom to scroll through the items and loan based on availability and condition of the item, leaving them with a more transparent and simpler system to loan equipment and also a simpler application for staff to track current and incoming loans.

6b)

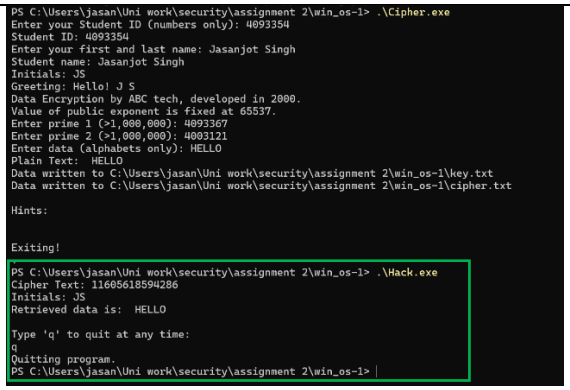
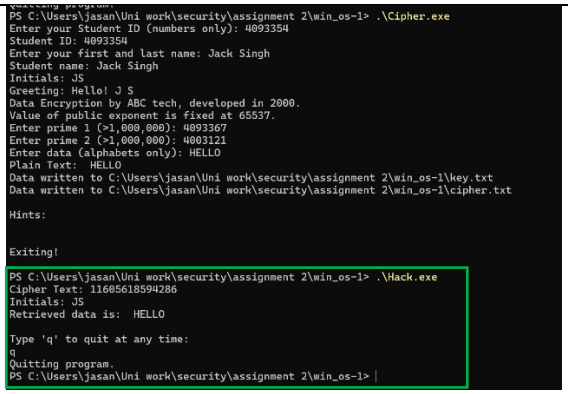
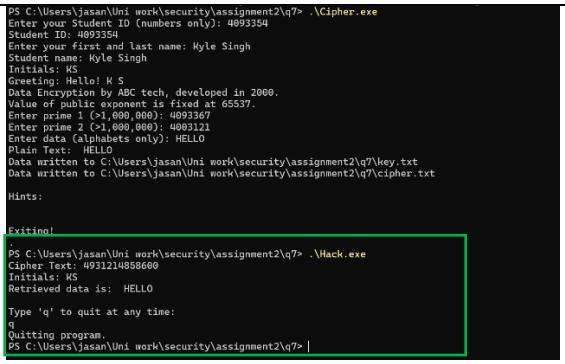
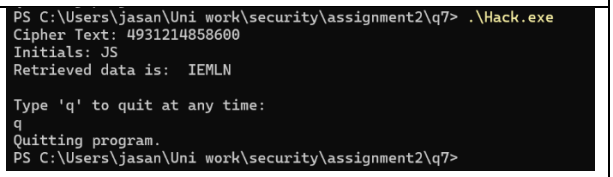
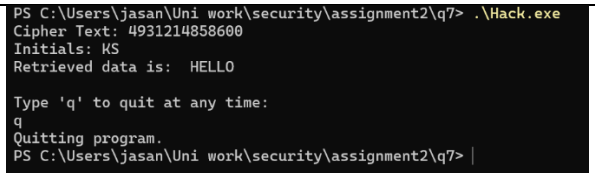
Side note: in case link does not work, the video is also in the assignment2 zip folder which was submitted under "**6b video demo.mp4"**

Call with Kyle Bonjour-20261018_112324-Meeting Recording.mp4

7.1)

I tested Cipher.exe and Hack.exe by changing 1 input at a time for each test conducted. Student ID, last name, the prime numbers (4093367 and 4003121) and plaintext (HELLO) all fixed while I changed the first name for tests 2 and 3 but then edited the stored initials for tests 4 and 5. The public key (65537, 16386243398407) remained the same throughout all tests.

Test	Changes made	Ciphertext	Output of Hack.exe

1	Default input, name was Jasanjot Singh	11605618594286	 **output: HELLO**
2	Changed name to Jack Singh	11605618594286	 **output: HELLO**
3	Changed name to Kyle Singh	4931214858600	 **output: HELLO**
4	Changed initials in cipher.txt to JS from KS	4931214858600	 **output: IEMLN**
5	Restored initials back to KS from JS	4931214858600	 **output: HELLO**

These results above show that changing the name while keeping same initials didn't affect the ciphertext, however changing the first initial had changed the ciphertext. Following that, changing the stored initials on the file itself had changed the recovered message and once original initials were restored, the original message was also recovered. The communication can be compromised because Hack.exe recovered the original message using the public key and ciphertext file without us supplying the private key. The initials had provided no secret protection as they were included in the ciphertext file. The small RSA modulus is another weakness, as recovering the factors 4093367 and 4003121 allows the RSA private key to be calculated, however these tests don't prove that factorisation is the method used by Hack.exe. Cipher.exe displayed statements about the company name, development year and fixed public exponent of 65537. My tests did not show that the company name or year affected the security and using 65537 as the public exponent is not a weakness by itself.

7.2)

The concatenated student ID was **40933544003110** and the immediate prime below this was **40933544003087** and the immediate prime above was **40933544003113**, these primes were entered into Cipher.exe and the public key (65537, 167555502465372397280960983 1) was produced. The plaintexts **HELLO** and **KYLE** were encrypted separately using the same primes.

Plaintext	Ciphertext	Hack.exe output
HELLO	72340156134195752708895478 8	PS C:\Users\jasan\Uni work\security\assignment2\q7> .\Cipher.exe Enter your Student ID (numbers only): 4093354 Student ID: 4093354 Enter your first and last name: Jasanjot Singh Student name: Jasanjot Singh Initials: JS Greeting: Hello! J S Data Encryption by ABC tech, developed in 2000. Value of public exponent is fixed at 65537. Enter prime 1 (>1,000,000): 40933544003087 Enter prime 2 (>1,000,000): 40933544003113 Enter data (alphabets only): HELLO Plain Text: HELLO Data written to C:\Users\jasan\Uni work\security\assignment2\q7\key.txt Data written to C:\Users\jasan\Uni work\security\assignment2\q7\cipher.txt Hints: Exiting! PS C:\Users\jasan\Uni work\security\assignment2\q7> .\Hack.exe Cipher Text: 72340156134195752708895478 8 Initials: JS Retrieved data is: HELLO Type 'q' to quit at any time: q Quitting program. PS C:\Users\jasan\Uni work\security\assignment2\q7> **Output: HELLO**
KYLE	32460664400991406696164590 0	PS C:\Users\jasan\Uni work\security\assignment2\q7> .\Cipher.exe Enter your Student ID (numbers only): 4093354 Student ID: 4093354 Enter your first and last name: Jasanjot Singh Student name: Jasanjot Singh Initials: JS Greeting: Hello! J S Data Encryption by ABC tech, developed in 2000. Value of public exponent is fixed at 65537. Enter prime 1 (>1,000,000): 40933544003087 Enter prime 2 (>1,000,000): 40933544003113 Enter data (alphabets only): KYLE Plain Text: KYLE Data written to C:\Users\jasan\Uni work\security\assignment2\q7\key.txt Data written to C:\Users\jasan\Uni work\security\assignment2\q7\cipher.txt Hints: Exiting! PS C:\Users\jasan\Uni work\security\assignment2\q7> .\Hack.exe Cipher Text: 32460664400991406696164590 0 Initials: JS Retrieved data is: KYLE Type 'q' to quit at any time: q Quitting program. PS C:\Users\jasan\Uni work\security\assignment2\q7> **Output: KYLE**

Keeping the primes fixed while changing the plaintext produced different ciphertexts, while the public key remained unchanged. Hack.exe successfully recovered both HELLO and KYLE, showing that recovery remained possible with these larger primes.

In Q7.1, Hack.exe recovered HELLO using smaller primes. In Q7.2, it also recovered HELLO using larger primes. Therefore, increasing prime size did not prevent recovery of the same plaintext. Under the tested conditions, larger primes alone were insufficient to address the observed security weakness.

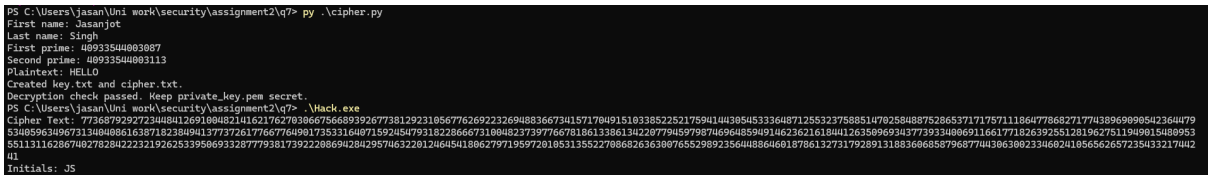
7.3)

Code has been written under the file name **“cipher.py”**

My program uses AES to encrypt the message with a random secret key. RSA-OAEP then protects that key. Unlike Q7.1, the public initials are not used as the encryption key. The program checks the entered primes but generates its own 2048-bit RSA key for encryption.

I tested HELLO and KYLE. Both passed the program’s decryption check, but Hack.exe did not recover either message during my tests. The screenshots show these results. This shows that Hack.exe could not recover the messages within the time tested.

Plaintext: HELLO



Plaintext: KYLE

